Discrete Variational Symmetrical Top: an Archetype for Rigid-Body Dynamics

Brian Beckman, December 2024, Revised January 2025

Abstract

We compare two numerical solutions for the free motion of a spinning, symmetrical top. One solution is by the standard method of integrating differential equations, discretizing the independent time variable as late as possible in the solution process. The other solution begins by discretizing the dynamical variables and the Lagrangian as early as possible in the solution process, then solving derived algebraic recurrences by root-finding.

Our contribution is to find that the early-discretized solution has no secular, long-term drift in conserved quantities (energy and momenta), unlike the standard, late-discretized solution, which exhibits small but steady drifts in the sixth or seventh significant figure.

While the early-discretized solution has no secular drifts, its conserved quantities have oscillatory noise in the fourth or fifth significant figure, about 2000 times larger in amplitude than the drift in the standard solution. Considering its other benefits—ease of implementation and tuning—we find that the discrete method is worth further study and optimization.

Introduction

Traditionally, one solves problems in classical mechanics via differential equations of motion, discretized for numerical solution as last as possible in the solution process. We call this tactic ***late dis-***

cretization. By contrast, one can discretize the dynamical variables and the Lagrangian at the outset and avoid differential equations and their numerical methods altogether. We call this tactic ***early discretization***. The resulting equations of motion are algebraic, solved by iterated root-finding. Theoretically, at least, the early-discretized methods conserve energy and momenta by construction.[1, 2, 3]

Numerical methods for differential equations comprise a vast variety of algorithms, sensitive to multiple tuning parameters. Bringing a solution to ground can be an expensive process of trial-and-error and compromises. By contrast, discrete methods have only one tuning parameter—the time step.

In this notebook, we contrast a traditional numerical solution for the symmetrical top with a new discrete solution. The two solutions match to three significant figures: within the width of a plotting line. The traditional solution exhibits secular drift in conserved energy and momenta. While small—in the sixth or seventh significant figure—the poor conservation will eventually become noticeable, requiring a “reboot.”. By contrast, the discrete solution exhibits only stable oscillatory error in the conserved quantities. While larger—in the fourth or fifth decimal place—these errors are not noticeable, mostly because they are stable in the long run.

The contribution of this notebook is applying discrete dynamics to this problem from scratch. While we have seen theoretical discrete dynamics for the top,[9] we have not seen a convincing numerical demonstration. The theory leaves us unsatisfied that the method can be effectively implemented, let alone easily. Here, we show that it can be both.

Despite these demonstrable benefits, the discrete implementation in this notebook is too slow. However, with this and our prior successes with discrete harmonic oscillators,[7, 8] we are convinced that the natural benefits of discrete dynamics justify further study and optimization.

References

1. Marsden and West, 2001, *Discrete Mechanics and Variational Integrators*, http://www.cds.caltech.edu/~marsden/bib/2001/09-MaWe2001/MaWe2001.pdf
2. Matthew West, *Variational Integrators*, Caltech Thesis, May 2004, https://thesis.library.caltech.edu/2492/1/west_thesis.pdf
3. Marin Kobilarov and Jerrold E. Marsden, *Discrete Geometric Optimal Control on Lie Groups*, IEEE Transactions on Robotics, Vol. 27, No. 4, August 2011.
4. https://hepweb.ucsd.edu/ph110b/110b_notes/node36.html
5. https://bingweb.binghamton.edu/~suzuki/Math-Physics/LN-13S_Symmetric_top.pdf (vetted against Reference 4 in a separate notebook)
6. William L. Burke, *Applied Differential Geometry*, Cambridge University Press; 1st edition (January 12, 2008)
7. Brian C. Beckman, *Harmonic Oscillator via Discrete Lagrangian*, https://community.wolfram.com/groups/-/m/t/3161109, 2024

8. Brian C. Beckman, *Discrete Damped Harmonic Oscillator*, https://community.wolfram.com/groups/-/m/t/3174169, 2024
9. A. I. Bobenko, Yu. B. Suris, *Discrete time Lagrangian mechanics on Lie groups, with an application to the Lagrange top*, https://arxiv.org/abs/solv-int/9810018v1, 1998

Outline

1. Set up the problem.
2. Numerically solve the continuous Euler-Lagrange equations, vetting Reference 4.
3. Explore constants of the motion. Observe numerical drift in energy and momenta.
4. Formulate and solve the discrete Lagrangian. Compare results with the continuous solution. Observe only stable oscillatory noise.

Problem Set-Up

There are four parameters or *free variables* for the problem. Because the top is symmetrical about its spinning axis, its moment-of-inertia tensor has only two independent components, free variables I_1 and I_3. Alternatively, we can model the top as an oblate spheroid with semi-major axis l and equatorial radius s as free variables.

The acceleration of gravity g and the mass m of the top are the other two free variables.

Continuous Equations of Motion

In[ ]:=
```
ClearAll[L, F, V, T, q, ω, ω2, ϕ, θ, ψ, ϕd, θd, ψd, I1, I3, l];
```

Lagrangian

Generalized Coordinates and Velocities

Choose generalized coordinates azimuth, ϕ, co-elevation, θ, and nodal angle about the spin axis, ψ, i.e., Euler angles in the x-convention (https://mathworld.wolfram.com/EulerAngles.html).

The corresponding generalized velocities are $\dot{\phi}$, $\dot{\theta}$, and $\dot{\psi}$, written as ϕd, θd, ψd because $\dot{\phi}$, $\dot{\theta}$, and $\dot{\psi}$ are not valid Mathematica syntax for variables.

- The standard notation $\partial L / \partial \dot{\phi}$ is a cruel pun. Consider the dedication of Reference 6, "*To all those who, like me, have wondered how in hell you can change* $\dot{q}$ *without changing* q."

- We can't take the derivative of L with respect to the time derivative $\dot{\phi} = \phi'[t]$ of a function $\phi[t]$. $\partial L / \partial \dot{\phi}$ really means $\partial L / \partial(\phi\text{d})$, where ϕd is an ordinary parameter variable of the arity-6 Lagrangian function $L[\phi, \theta, \psi, \phi\text{d}, \theta\text{d}, \psi\text{d}]$.
- Reference 6 develops the differential-geometry background, from a physicist's perspective, necessary for the theoretical developments in References 1, 2, 3, and 9. That's all we have to say about theory in this notebook.

Potential Energy

By inspection of the height l of the CG (Center of Gravity) from the contact point:

In[ ]:=
```
ClearAll[V];
V[X : {ϕ_, θ_, ψ_, ϕd_, θd_, ψd_}] := m l g Cos[θ];
With[{X = {ϕ, θ, ψ, ϕd, θd, ψd}},
 V[X]]
```

Out[ ]=
```
g l m Cos[θ]
```

Checked

Units of measure: mass * length^2 / time^2 = energy.

Intuition check: when θ is small, the top is upright and potential energy is high, just like cosine.

The force on the top is

In[ ]:=
```
With[{X = {ϕ, θ, ψ, ϕd, θd, ψd}},
 -D[V[X], θ]]
```

Out[ ]=
```
g l m Sin[θ]
```

zero at the pole, then acting in the direction of increasing θ, that is, vertically downwards, and larger with larger θ.

Kinetic Energy

Consider an inertial reference frame instantaneously oriented with the current Euler angles of the top, but not rotating with the top. Measure the overall angular-velocity vector of the top in this frame in terms of the time derivatives of the angles. Write kinetic energy in terms of that angular-velocity vector in that frame. Denote the 6-tuple $\{\phi, \theta, \psi, \phi\text{d}, \theta\text{d}, \psi\text{d}\}$ as the ***state vector*** X. The [[1, 1]] notation in the kinetic-energy formula merely fetches the scalar value from the one-and-only slot in the matrix formula.

In[ ]:=
```
ClearAll[M, L, T];
M = ( I1  0  0
      0  I1  0
      0   0  I3 );
ω[{ϕ_, θ_, ψ_, ϕd_, θd_, ψd_}] := {
    {ϕd Sin[θ] Sin[ψ] + θd Cos[ψ]},
    {ϕd Sin[θ] Cos[ψ] - θd Sin[ψ]},
    {ϕd Cos[θ] + ψd}};
T[X : {ϕ_, θ_, ψ_, ϕd_, θd_, ψd_}] := (1/2 ω[X]ᵀ.M.ω[X])[[1, 1]];
L[X : {ϕ_, θ_, ψ_, ϕd_, θd_, ψd_}] := T[X] - V[X];
```

Checked

Check *T* against Reference 4 (*Mathematica* does not automatically reduce *T* to the right-hand side, and note that === does not work for unknown reasons):

In[ ]:=
```
With[{X = {ϕ, θ, ψ, ϕd, θd, ψd}},
 (T[X] == (1/2 I1 (θd^2 + Sin[θ]^2 ϕd^2) + 1/2 I3 (Cos[θ] ϕd + ψd)^2)) // FullSimplify]
```

Out[ ]=
```
True
```

Check our *L* against *L* hand-copied from Reference 4.

In[ ]:=
```
((1/2 I1 (ϕd^2 Sin[θ]^2 + θd^2) + 1/2 I3 (ϕd Cos[θ] + ψd)^2 - m l g Cos[θ])
   == With[{X = {ϕ, θ, ψ, ϕd, θd, ψd}}, L[X]]) // FullSimplify
```

Out[ ]=
```
True
```

Euler-Lagrange Equations

Writing $\partial L/\partial v$ instead of the pun $\partial L\big/\partial \dot{q}$, expand

$$\frac{d}{dt}\frac{\partial L}{\partial v}-\frac{\partial L}{\partial q}=0 \tag{1}$$

as v ranges over $\{\phi\mathrm{d}, \theta\mathrm{d}, \psi\mathrm{d}\}$ and q ranges in lock-step over $\{\phi, \theta, \psi\}$.

Here is $\frac{\partial L}{\partial q}$ as a column vector. Note the zeros.

In[ ]:=
```
ClearAll[dLdq];
(dLdq = (With[{X = {ϕ, θ, ψ, ϕd, θd, ψd}},
      Table[D[L[X], q], {q, {ϕ, θ, ψ}}]] // FullSimplify)) // MatrixForm
```

Out[ ]//MatrixForm=

$$\begin{pmatrix} 0 \\ (g\,l\,m - I3\,\phi d\,\psi d + (I1 - I3)\,\phi d^2\,\mathrm{Cos}[\theta])\,\mathrm{Sin}[\theta] \\ 0 \end{pmatrix}$$

See dimensions of energy—mass * length^2/time^2—because ϕ, θ, and ψ, are dimensionless.

Here is $\frac{\partial L}{\partial v}$:

In[ ]:=
```
ClearAll[dLdqDot];
(dLdqDot = (With[{X = {ϕ, θ, ψ, ϕd, θd, ψd}},
      Table[D[L[X], qd], {qd, {ϕd, θd, ψd}}]] // FullSimplify)) // MatrixForm
```

Out[ ]//MatrixForm=

$$\begin{pmatrix} I3\,\psi d\,\mathrm{Cos}[\theta] + \frac{1}{2}\,\phi d\,(I1 + I3 + (-I1 + I3)\,\mathrm{Cos}[2\,\theta]) \\ I1\,\theta d \\ I3\,(\psi d + \phi d\,\mathrm{Cos}[\theta]) \end{pmatrix}$$

Now see dimensions of mass * length^2/time rather than mass * length^2/time^2. The second factor of time in the denominator returns via d/dt.

Conjugate Momenta

For any Lagrangian, $\frac{\partial L}{\partial \dot{q}}$ (pun intended) are the ***conjugate momenta*** p_q. In our case, the first, $p_\phi = \frac{\partial L}{\partial \dot{\phi}}$, and the third, $p_\psi = \frac{\partial L}{\partial \dot{\psi}}$, are ***conserved momenta***, or ***constants of the motion***. $\frac{\partial L}{\partial \phi}$ and $\frac{\partial L}{\partial \psi}$ are zero, thus so are $\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\phi}}\right) = \frac{dp_\phi}{dt}$ and $\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\psi}}\right) = \frac{dp_\psi}{dt}$ by the Euler-Lagrange equations; p_ϕ and p_ψ must be constant because their time derivatives vanish. This is an application of Noether's Theorem.

Reference 5 exhibits alternative, first-order equations of motion via these constants. We have, in a separate notebook, shown that References 4 and 5 are mutually consistent.

Checked

Manually check against Reference 4:

In[ ]:=
```
(dLdqDot[[1]] == ((I1 Sin[θ]^2 + I3 Cos[θ]^2) ϕd + I3 ψd Cos[θ])) // FullSimplify
```

Out[ ]=

```
True
```

In[ ]:=
```
(dLdqDot[[3]] == (I3 (ψd + ϕd Cos[θ])))
```
Out[ ]=
```
True
```

Conjugate Momenta as Functions of State

Define functions of the state X for the conjugate momenta. These will be valuable later for numerical comparisons.

In[ ]:=
```
ClearAll[pϕ, pψ];
pϕ[X : {ϕ_, θ_, ψ_, ϕd_, θd_, ψd_}] := ((I1 Sin[θ]^2 + I3 Cos[θ]^2) ϕd + I3 ψd Cos[θ]);
pψ[X : {ϕ_, θ_, ψ_, ϕd_, θd_, ψd_}] := (I3 (ψd + ϕd Cos[θ]));
```

Second-Order Equations of Motion

m, I_1, and I_3 are constants. Rewrite their total time derivatives to zero (it's interesting in some problems to let them vary with time, but not here)

In[ ]:=
```
ClearAll[constRules];
constRules = {Dt[m, t] → 0, Dt[I1, t] → 0, Dt[I3, t] → 0};
```

Total time derivatives of the conjugate momenta yield second-order terms:

In[ ]:=
```
ClearAll[ddLdqDotdt];
(ddLdqDotdt = (Dt[dLdqDot, t] /. constRules // FullSimplify)) // MatrixForm
```
Out[ ]//MatrixForm=
```
(1/2 (I1 + I3 + (-I1 + I3) Cos[2 θ]) Dt[ϕd, t] + I3 Cos[θ] Dt[ψd, t] - (I3 ψd + 2 (-I1 + I3) ϕd
                                                                    I1 Dt[θd, t]
                                  I3 (Cos[θ] Dt[ϕd, t] + Dt[ψd, t] - ϕd Dt[θ, t] Sin[θ])
```

Finally, the equations of motion, $\frac{d}{dt}\frac{\partial L}{\partial \dot{q}} - \frac{\partial L}{\partial q} = 0$:

In[ ]:=
```
ClearAll[hardEqns];
(hardEqns = Table[(lhs == 0) // FullSimplify, {lhs, ddLdqDotdt - dLdq}]) //
 MatrixForm
```
Out[ ]//MatrixForm=
```
(1/2 (I1 + I3 + (-I1 + I3) Cos[2 θ]) Dt[ϕd, t] + I3 Cos[θ] Dt[ψd, t] == (I3 ψd + 2 (-I1 + I3) ϕc
                        I1 Dt[θd, t] + (-g l m + I3 ϕd ψd + (-I1 + I3) ϕd^2 Cos[θ]) Sin[θ] ==
                            I3 (Cos[θ] Dt[ϕd, t] + Dt[ψd, t]) == I3 ϕd Dt[θ, t] Sin[θ]
```

Set up symbolic rewrites of the equations of motion for numerical solution via `NDSolve`:

In[ ]:=
```
ClearAll[fTimeRules];
fTimeRules = {
   Dt[ϕd, t] → ϕ''[t],
   Dt[θd, t] → θ''[t],
   Dt[ψd, t] → ψ''[t],
   ϕd → ϕ'[t],
   θd → θ'[t],
   ψd → ψ'[t],
   ϕ → ϕ[t],
   θ → θ[t],
   ψ → ψ[t]};
```

It's not easy to interpret, or even believe, these equations straight-up, but the numerical solutions deliver the goods.

In[ ]:=
```
(hardEqns /. fTimeRules) // MatrixForm
```

Out[ ]//MatrixForm=

$$\begin{pmatrix} \frac{1}{2}\,(I1 + I3 + (-I1 + I3)\,\mathrm{Cos}[2\,\theta[t]])\,\phi''[t] + I3\,\mathrm{Cos}[\theta[t]]\,\psi''[t] == \mathrm{Sin}[\theta[t]]\,\theta'[t]\,(2\,(-I1 \\ \mathrm{Sin}[\theta[t]]\,\left(-g\,l\,m + (-I1 + I3)\,\mathrm{Cos}[\theta[t]]\,\phi'[t]^2 + I3\,\phi'[t]\,\psi'[t]\right) + I \\ I3\,(\mathrm{Cos}[\theta[t]]\,\phi''[t] + \psi''[t]) == I3\,\mathrm{Sin}[\theta[t]]\,\theta'[t]\,\phi'[t] \end{pmatrix}$$

Numerical Solution

Pick numerical values for the free variables. The "4" reminds us we're working with Reference 4. In a later notebook, we reconcile these values with Reference 5.

In[ ]:=
```
ClearAll[numRules4];
numRules4 = {g → 1, m → 1, l → 2, I1 → 2, I3 → 1};
(hardEqns /. fTimeRules /. numRules4) // MatrixForm
```

Out[ ]//MatrixForm=

$$\begin{pmatrix} \frac{1}{2}\,(3 - \mathrm{Cos}[2\,\theta[t]])\,\phi''[t] + \mathrm{Cos}[\theta[t]]\,\psi''[t] == \mathrm{Sin}[\theta[t]]\,\theta'[t]\,(-2\,\mathrm{Cos}[\theta[t]]\,\phi'[t] + \psi'[t \\ \mathrm{Sin}[\theta[t]]\,\left(-2 - \mathrm{Cos}[\theta[t]]\,\phi'[t]^2 + \phi'[t]\,\psi'[t]\right) + 2\,\theta''[t] == 0 \\ \mathrm{Cos}[\theta[t]]\,\phi''[t] + \psi''[t] == \mathrm{Sin}[\theta[t]]\,\theta'[t]\,\phi'[t] \end{pmatrix}$$

Change `tN` if you want a longer integration.

In[ ]:=
```
ClearAll[tN];
tN = 70;
```

Mathematica has no trouble, numerically, with these equations, given plausible initial conditions. There is a great deal of automation hidden in this call: automatic (and opaque) choice of algorithm, time step, and values for various tuning parameters. If we were writing an integrator from scratch, we would have much work to do.

In[ ]:=
```
ClearAll[nsoln];
nsoln = NDSolve[(hardEqns /. fTimeRules /. numRules4) ~ Join ~ {
    ϕ[0] == 0, ϕ'[0] == 0,
    θ[0] == 42 °, θ'[0] == 0,
    ψ[0] == 0, ψ'[0] == 5},
  {ϕ[t], θ[t], ψ[t]},
  {t, 0, tN}]
```

Out[ ]=

{{ϕ[t] → InterpolatingFunction[Domain: {{0., 70. }} Output: scalar][t],

θ[t] → InterpolatingFunction[Domain: {{0., 70. }} Output: scalar][t],

ψ[t] → InterpolatingFunction[Domain: {{0., 70. }} Output: scalar][t]}}

Conservation of Energy

Write the energy $E_4[X]$ as a function of state X. The subscript "4" again reminds us that we're checking Reference 4.

In[ ]:=
```
ClearAll[E4];
E4[X : {ϕ_, θ_, ψ_, ϕd_, θd_, ψd_}] := T[X] + V[X];
```

In[ ]:=
```
With[{X = {ϕ[t], θ[t], ψ[t], ϕ'[t], θ'[t], ψ'[t]}},
   E4[X]] /. numRules4 // FullSimplify
```

Out[ ]=

$$2\,\mathrm{Cos}[\theta[t]] + \theta'[t]^2 - \frac{1}{4}\,(-3 + \mathrm{Cos}[2\,\theta[t]])\,\phi'[t]^2 + \mathrm{Cos}[\theta[t]]\,\phi'[t]\,\psi'[t] + \frac{1}{2}\,\psi'[t]^2$$

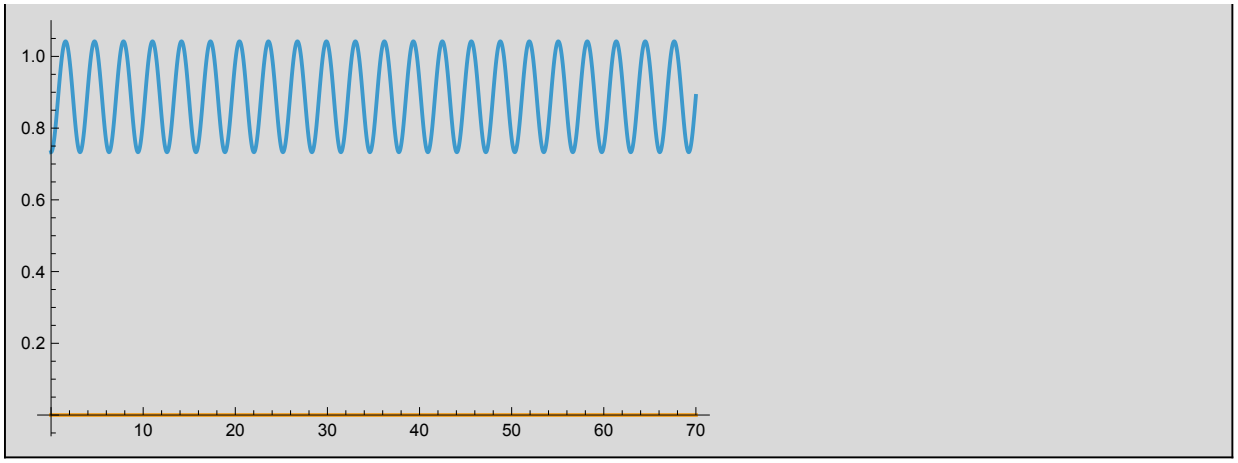
Funny Plotting Business

We need time derivatives of the angles to plot energy and momentum. It's not immediately straightforward to compute derivatives of `NDsolve`'s results, which are not functions, rather expressions depending on the hard-coded parameter t. Mathematica's `Derivative` function requires functions, not expressions.

Convert the expressions to functions. Also, avoid the `Symbol` t, even if encapsulated in `Modules` or as dummy variables in `Plot`. To wit, the following silently failed:

In[ ]:=
```
Module[{fθ},
 fθ[u_] := ((θ[t] /. nsoln) /. {t → u});
 Plot[{fθ[t], fθ'[t]}, {t, 0, tN}]]
```

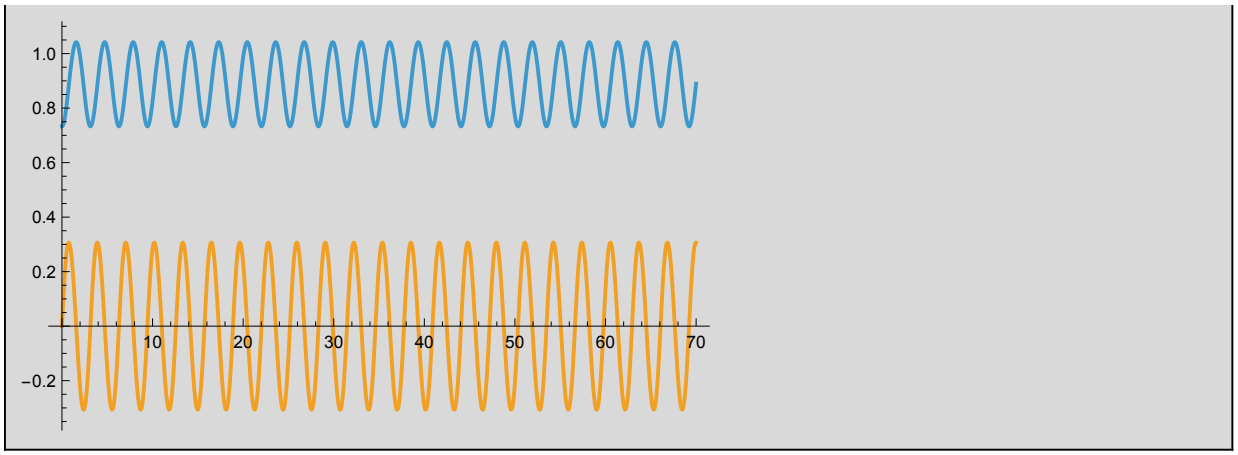
Out[ ]=



The following DOES work, so long as the dummy variable in `Plot` is anything *except* `t`:

In[ ]:=
```
Module[{fθ},
 fθ[u_] := ((θ[t] /. nsoln[[1]]) /. {t → u});
 Plot[{fθ[u], fθ'[u]}, {u, 0, tN}]]
```

Out[ ]=



We're now equipped to compute energy and momenta.

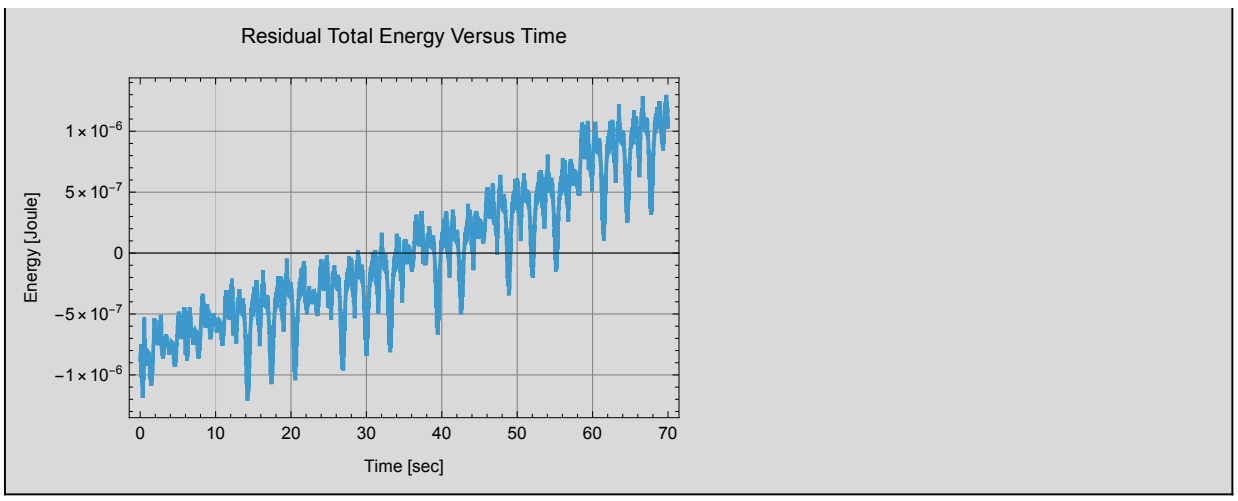
Checked

Energy, about 14 Joule, is conserved to five figures. It is, however, trending upwards due to numerical effects, which we will not see in the discrete solution.

In[ ]:=
```
ClearAll[displayResidualEnergy];
displayResidualEnergy[Efunction_, label_] :=
  Module[{fϕ, fθ, fψ, X, meanEf},
   fϕ[u_] := ((ϕ[t] /. nsoln⟦1⟧) /. {t → u});
   fθ[u_] := ((θ[t] /. nsoln⟦1⟧) /. {t → u});
   fψ[u_] := ((ψ[t] /. nsoln⟦1⟧) /. {t → u});
   X[u_] := {fϕ[u], fθ[u], fψ[u], fϕ'[u], fθ'[u], fψ'[u]};
   (* mean Energy of the entire run *);
   meanEf =
    Echo@Mean[Table[(Efunction[X[u]] /. numRules4), {u, 0, tN, (tN - 0) / 100}]];
   Plot[(Efunction[X[u]] /. numRules4) - meanEf, {u, 0, tN},
    FrameLabel → {{"Energy [Joule]", ""}, {"Time [sec]", ""}},
    PlotLabel → label, Frame → Automatic, GridLines → Automatic]];
displayResidualEnergy[E4, "Residual Total Energy Versus Time"]
```

» 13.9863

Out[ ]=

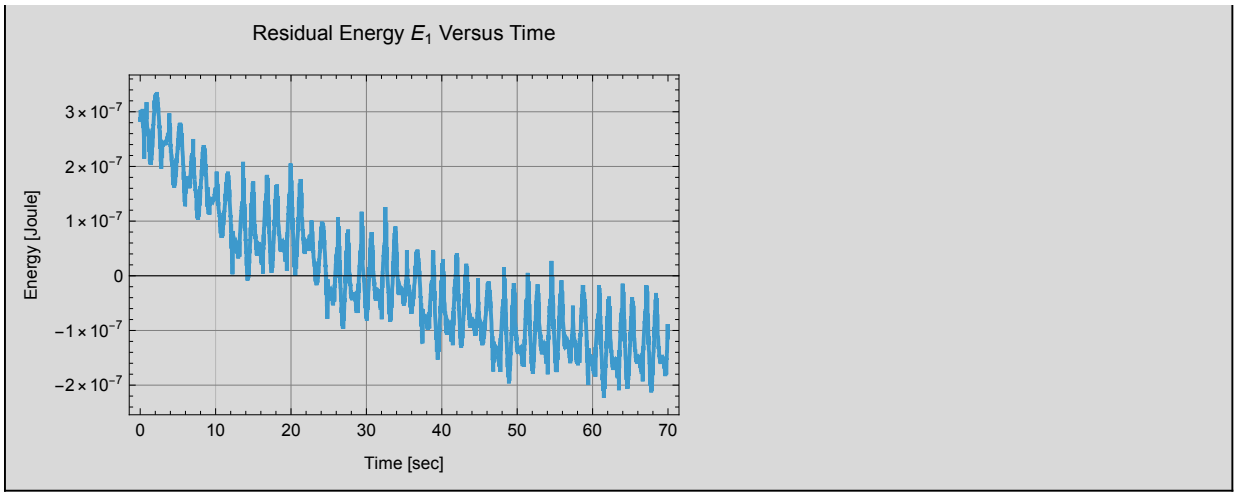


It turns out that two addends of energy are separately conserved, again with small, pesky drifts. The first and smaller, E_1, is the potential energy plus the kinetic energy of nutation angle θ plus the kinetic energy due to the (smaller) precession component, $\phi d \sin \theta$, perpendicular to the instantaneous axis of rotation:

In[ ]:=
```
ClearAll[E1];
E1[X : {ϕ_, θ_, ψ_, ϕd_, θd_, ψd_}] := m g l Cos[θ] + 1/2 I1 (θd^2 + ϕd^2 Sin[θ]^2);
displayResidualEnergy[E1, "Residual Energy E1 Versus Time"]
```

» 1.48629

Out[ ]=

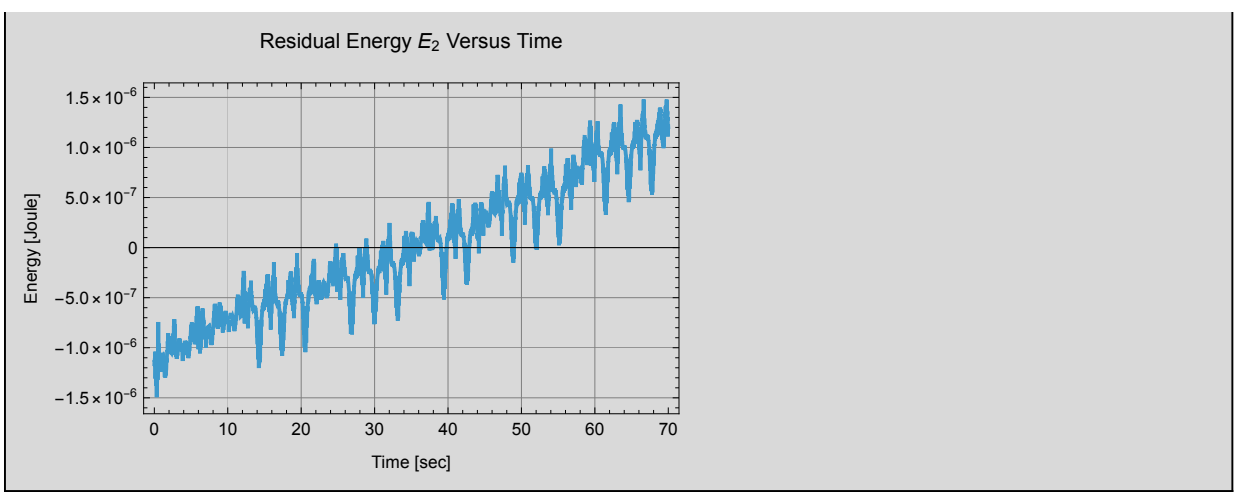


The second and larger conserved energy, E_2, is the kinetic energy of spin plus the kinetic energy due to the (larger) precession component, ϕd cos θ, parallel to the instantaneous axis of rotation:

In[ ]:=
```
ClearAll[E2];
E2[X : {ϕ_, θ_, ψ_, ϕd_, θd_, ψd_}] := 1/2 I3 (ϕd Cos[θ] + ψd)^2;
displayResidualEnergy[E2, "Residual Energy E2 Versus Time"]
```

» 12.5

Out[ ]=



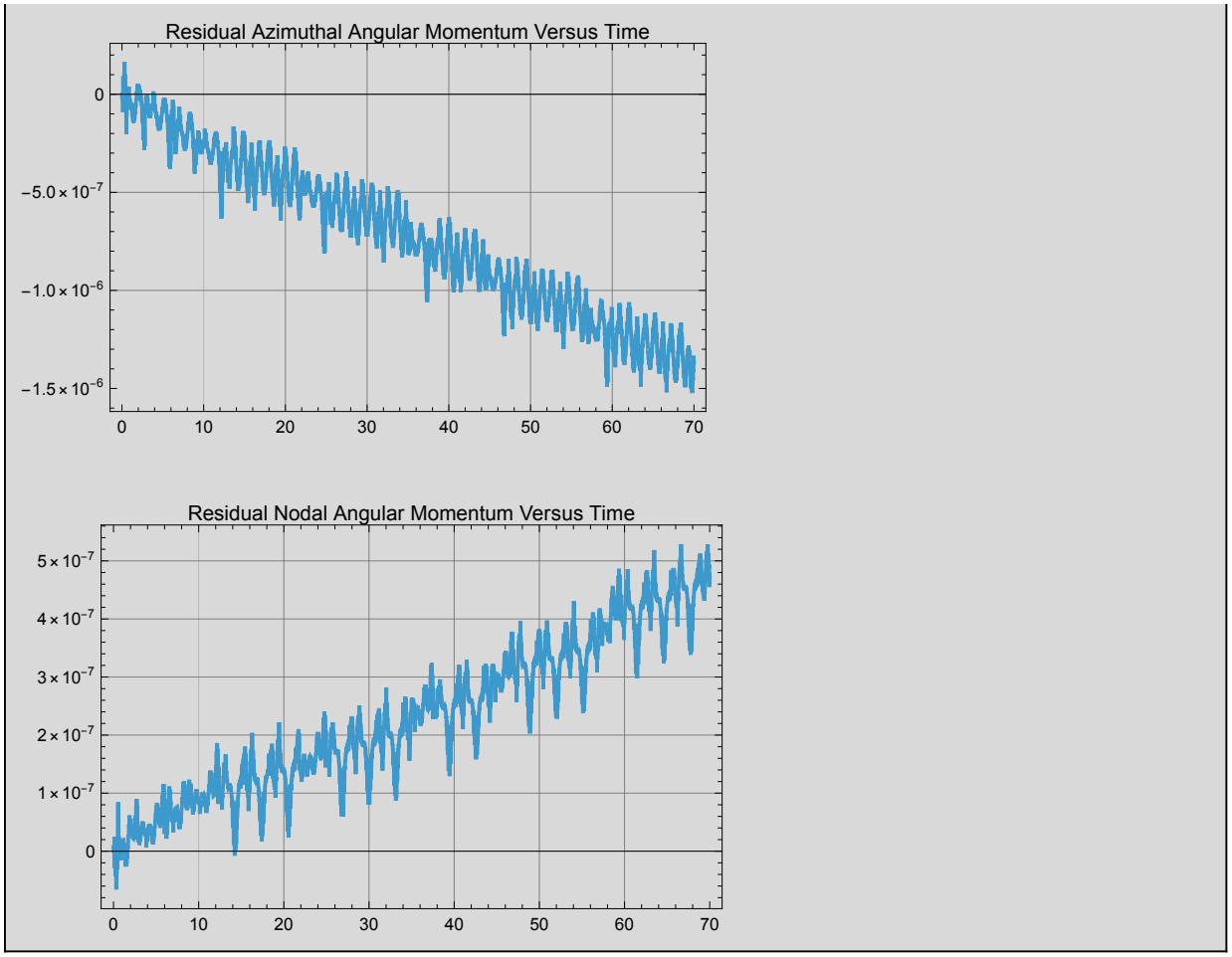
Conservation of Momenta

Check conservation of azimuthal p_ϕ and nodal p_ψ momenta via the functional forms defined above. See conservation within six figures with opposite drifts.

In[ ]:=
```
Module[{fϕ, fθ, fψ, X, meanPϕ, meanPψ},
 fϕ[u_] := ((ϕ[t] /. nsoln⟦1⟧) /. {t → u});
 fθ[u_] := ((θ[t] /. nsoln⟦1⟧) /. {t → u});
 fψ[u_] := ((ψ[t] /. nsoln⟦1⟧) /. {t → u});
 X[u_] := {fϕ[u], fθ[u], fψ[u], fϕ'[u], fθ'[u], fψ'[u]};
 meanPϕ = Echo@Mean[Table[pϕ[X[u]] /. numRules4, {u, 0, tN, 100}]];
 meanPψ = Echo@Mean[Table[pψ[X[u]] /. numRules4, {u, 0, tN, 100}]];
 Row[{
   Plot[(pϕ[X[u]] /. numRules4) - meanPϕ, {u, 0, tN}, ImageSize → Medium,
    PlotLabel → "Residual Azimuthal Angular Momentum Versus Time",
    Frame → Automatic, GridLines → Automatic],
   Plot[(pψ[X[u]] /. numRules4) - meanPψ, {u, 0, tN}, ImageSize → Medium,
    PlotLabel → "Residual Nodal Angular Momentum Versus Time",
    Frame → Automatic, GridLines → Automatic]}]]
```

```
» 3.71572
» 5.
```

Out[ ]=



Interactive Exploration

`NDSolve` is so fast that we can put it in a `Manipulate` to play around with initial conditions and integration time.

Graphics helpers:

In[ ]:=
```
ClearAll[jack, eulerRotate];
With[{o = {0, 0, 0},
  e1 = {1, 0, 0}, e2 = {0, 1, 0}, e3 = {0, 0, 1},
  tip = {0, 0, 1}, pr = 1.25},
 eulerRotate[obj_, ϕ_, θ_, ψ_] :=
  With[{r3 = RotationMatrix[ϕ, e3]},
   With[{e1Prime = r3.e1},
    With[{e3DoublePrime =
       RotationMatrix[θ, e1Prime].r3.e3},
     Rotate[
      Rotate[
       Rotate[
        obj,
        ϕ, e3],
       θ, e1Prime],
      ψ, e3DoublePrime]]]];
 jack[opacity_ : 0.1, diameter_ : 0.01] := {
   Opacity[opacity],
   {Red, Arrow[Tube[{o, e1}, diameter]]},
   {Darker[Green], Arrow[Tube[{o, e2}, diameter]]},
   {Blue, Arrow[Tube[{o, e3}, diameter]]},
   {Lighter[Cyan], Arrow[Tube[{o, -e1}, diameter]]},
   {Magenta, Arrow[Tube[{o, -e2}, diameter]]},
   {Yellow, Arrow[Tube[{o, -e3}, diameter]]}}]
```

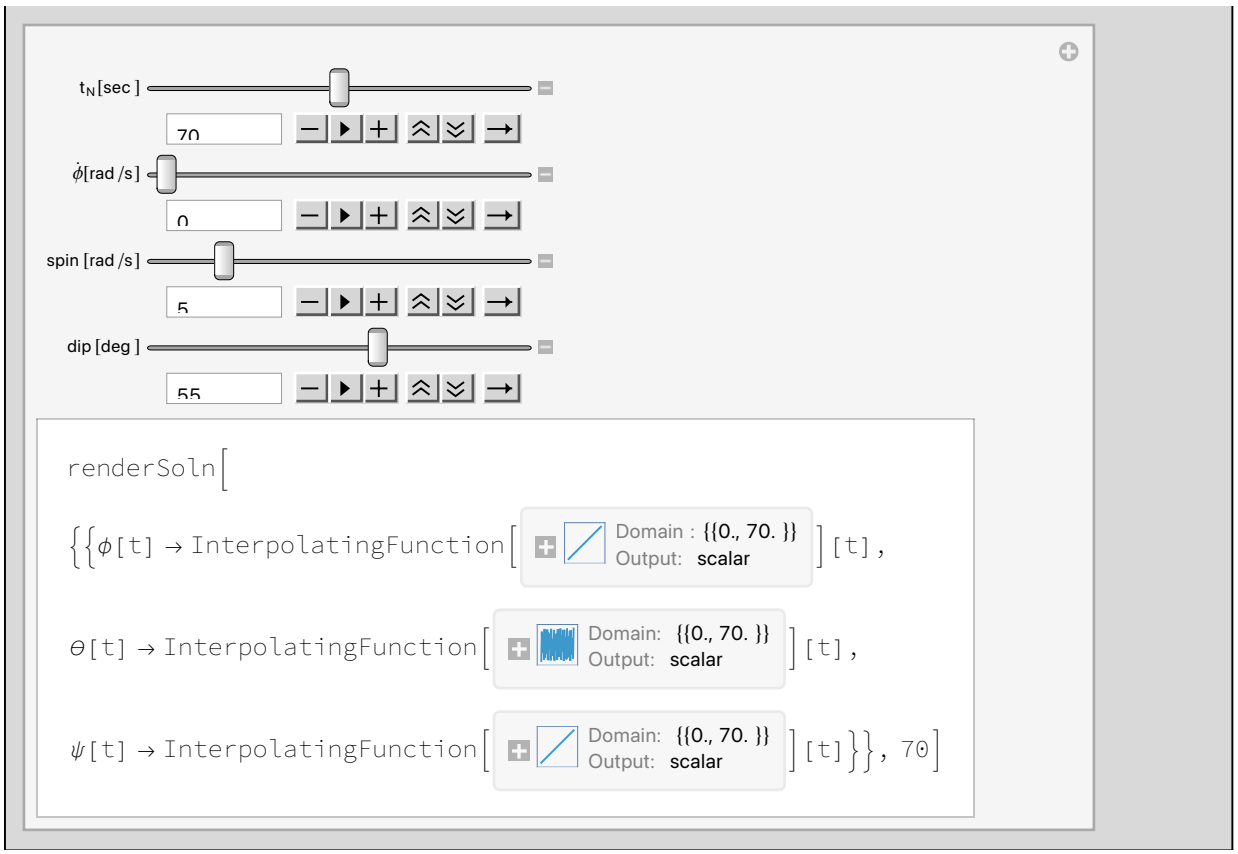
A rendering function:

In[ ]:=
```
ClearAll[renderSoln];
renderSoln[nsoln_, tn_] :=
  With[{pr = 1.25, e1 = {1, 0, 0}, e2 = {0, 1, 0}, e3 = {0, 0, 1}, squish = 0.0001},
   With[{prl = {{-pr, pr}, {-pr, pr}, {-pr, pr}}},
    Show[
     Graphics3D[{
       jack[.25, 0.0125],
       Opacity[0.1], Sphere[],
       Opacity[0.05],
       Blue, Cylinder[squish {-e3, e3}],
       Red, Cylinder[squish {-e1, e1}],
       Green, Cylinder[squish {-e2, e2}]}],
     ParametricPlot3D[{
        Sin[θ[t]] Cos[ϕ[t]],
        Sin[θ[t]] Sin[ϕ[t]],
        Cos[θ[t]]} /. nsoln,
      {t, 0, tn}, PlotRange → prl, AspectRatio → {1, 1, 1}]]]];
```

Here is a playground with manipulable initial spin speed ψd, precession throw ϕd, and dip angle θ_0. See the obvious nutation and precession.

Amongst interesting exercises, check that, with no initial spin, the top just falls over, i.e., rotates around the y axis because we don't have a "hard floor". This presages the inverted spherical pendulum, the next problem in this series of notebooks on discrete dynamics.

In[ ]:=

```
With[{initials = {ϕ[0] == 0 °, θ'[0] == 0, ψ[0] == 0 °},
  eqns = (hardEqns /. fTimeRules /. numRules4)},
 Manipulate[
  Module[{nsoln = NDSolve[
      eqns ~ Join ~ initials ~ Join ~ {
        ψ'[0] == ψd0, θ[0] == θ0 °, ϕ'[0] == ϕd0},
      {ϕ[t], θ[t], ψ[t]},
      {t, 0, tn}]},
   renderSoln[nsoln, tn]],
  {{tn, 70, "tN[sec]"}, 0, 140, Appearance → "Open"},
  {{ϕd0, 0, "ϕ̇[rad/s]"}, 0, 3, Appearance → "Open"},
  {{ψd0, 5, "spin[rad/s]"}, 0, 30, Appearance → "Open"},
  {{θ0, 42, "dip[deg]"}, 0, 90, Appearance → "Open"}]]
```

Out[ ]=